

Seascape Ecology

Lab 08 - Seascape Change

Ryan Reisinger

2025-12-02

Introduction

In this tutorial we'll look at a few fairly simple ways of analysing change in seascape data. We will assume a simple case, where we examine change in discrete time. That is, at fixed time points or intervals (time slices). In Chapter 6 of Pittman, Jackson et al. discuss some more complicated ways of modelling change, but for this lab we'll keep it simple.

Remember, we have two different conceptual models for representing seascape patterns: 1) the continuous gradient model, and 2) the patch-mosaic model. We will look at change in continuous gradient models here. Change in patch-mosaic models is covered in Lab 9.

Change in continuous gradient models

Data

As our example here, we'll use data from the paper “**Spatiotemporal Overlap of Baleen Whales and Krill Fisheries in the Western Antarctic Peninsula Region**” (Reisinger et al. 2022; <https://doi.org/10.3389/fmars.2022.914726>). In the paper, we looked at the spatiotemporal overlap between baleen whales (minke and humpback whales) and the krill fishery in the Western Antarctic Peninsula. The krill fishery has become more spatially concentrated over the last few years, raising concerns about the impact of local krill depletion on predators (whales, seals, and seabirds).

The simplest case: two points in time

In the simplest case, we have only two points in time (or two ‘timeslices’). Hence, we are comparing a pair of rasters in some way.

Let's read in some data on Antarctic krill fishing catch in the Western Antarctic Peninsula region. We'll read the data in directly from the Github repository associated with the paper mentioned above, Reisinger et al. (2022) from the Github repository here: <https://github.com/ryanreisinger/whaleKrillOverlap>

First we load our libraries. Remember, you'll have to install some of these packages if you don't already have them.

```
library(terra)
```

```
## terra 1.8.70
```

```
library(ggplot2)
```

```
library(tidyterra)
```

```
##
```

```
## Attaching package: 'tidyterra'
```

```
## The following object is masked from 'package:stats':
##
##   filter
```

```
library(dplyr)
```

```
##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:terra':
##
##   intersect, union

## The following objects are masked from 'package:stats':
##
##   filter, lag

## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union
```

```
library(tidyr)
```

```
##
## Attaching package: 'tidyr'

## The following object is masked from 'package:terra':
##
##   extract
```

Then, we can load the files.

```
# 2016 season
fish_2016 <- rast("https://github.com/ryanreisinger/whaleKrillOverlap/raw/main/data_out/ccamlr_fishing_rasters_monthly/2016.tif")
# 2020 season
fish_2020 <- rast("https://github.com/ryanreisinger/whaleKrillOverlap/raw/main/data_out/ccamlr_fishing_rasters_monthly/2020.tif")
```

If you weren't able to read the files directly into R using the `rast` function, you can download the files from the Github repository, using the links below, and read them in from your local computer. You'll need to change the path to the file to match where you saved it. https://github.com/ryanreisinger/whaleKrillOverlap/raw/main/data_out/ccamlr_fishing_rasters_monthly/2020.tif https://github.com/ryanreisinger/whaleKrillOverlap/raw/main/data_out/ccamlr_fishing_rasters_monthly/2016.tif

Let's take a look at the rasters, noting their characteristics.

```
fish_2016
```

```
## class      : SpatRaster
## size       : 140, 200, 8  (nrow, ncol, nlyr)
## resolution : 0.1, 0.1  (x, y)
## extent     : -70, -50, -74, -60  (xmin, xmax, ymin, ymax)
## coord. ref.: lon/lat WGS 84 (EPSG:4326)
## source     : 2016.tif
## names      : 2016_1, 2016_2, 2016_3, 2016_4, 2016_5, 2016_6, ...
## min values :      0,      0.0,      0.0,      0.0,      0,      0, ...
## max values :      0, 347772.4, 466468.8, 973815.4, 1570922, 1710344, ...
```

```
fish_2020
```

```
## class      : SpatRaster
## size       : 140, 200, 8  (nrow, ncol, nlyr)
```

```
## resolution : 0.1, 0.1 (x, y)
## extent : -70, -50, -74, -60 (xmin, xmax, ymin, ymax)
## coord. ref. : lon/lat WGS 84 (EPSG:4326)
## source : 2020.tif
## names : 2020_1, 2020_2, 2020_3, 2020_4, 2020_5, 2020_6, ...
## min values : 0, 0, 0, 0, 0, 0, ...
## max values : 0, 0, 0, 1454277, 2220534, 3735528, ...
```

We first check the coordinate reference system:

```
crs(fish_2016)
```

```
## [1] "GEOGCRS[\"WGS 84\", \n      ENSEMBLE[\"World Geodetic System 1984 ensemble\", \n      MEMBER[\"Wo
```

It's WGS 1984, with EPSG code 4326 (<https://www.epsg.io>). You can see the EPSG code in the last slot of the above output, called 'ID'.

You could get the EPSG code directly like this:

```
crs(fish_2016, describe = TRUE)$code
```

```
## [1] "4326"
```

Notice that the spatial extent (longitude, latitude) is:

```
ext(fish_2016)
```

```
## SpatExtent : -70, -50, -74, -60 (xmin, xmax, ymin, ymax)
```

And the resolution (degrees) is:

```
res(fish_2016)
```

```
## [1] 0.1 0.1
```

You may have noticed that the dimensions of the raster are:

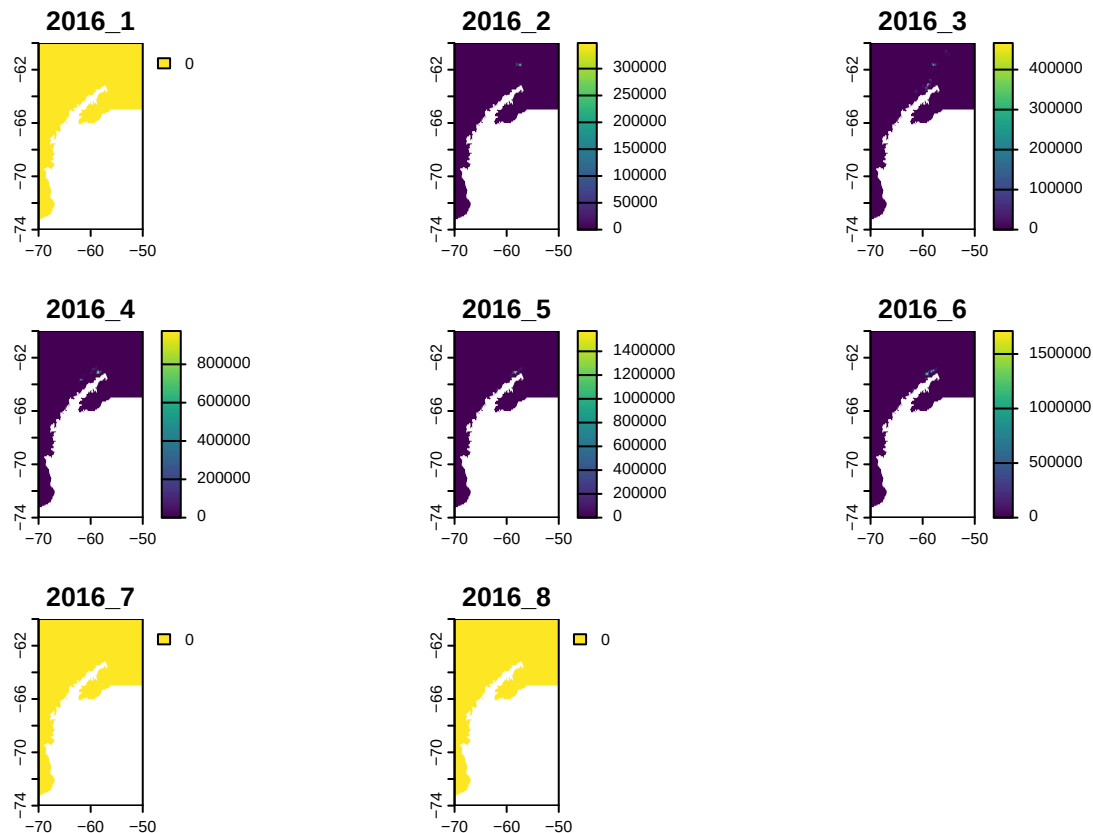
```
dim(fish_2016)
```

```
## [1] 140 200 8
```

The first values are the number of rows and columns, respectively, while the third value (8) is the number of layers. That means this raster is a 'stack' with 8 layers.

We can see that when we plot it.

```
plot(fish_2016)
```



In this case, each layer represents a month in the annual fishing season that runs from December (layer 1) to July the next year (layer 8). The values are the catch of Antarctic krill, in kg, in each pixel. Mostly you'll see zero catch; the catches are concentrated in small hotspots off the northern part of the Antarctic Peninsula. There is no catch at all in the first layer and the last two layers.

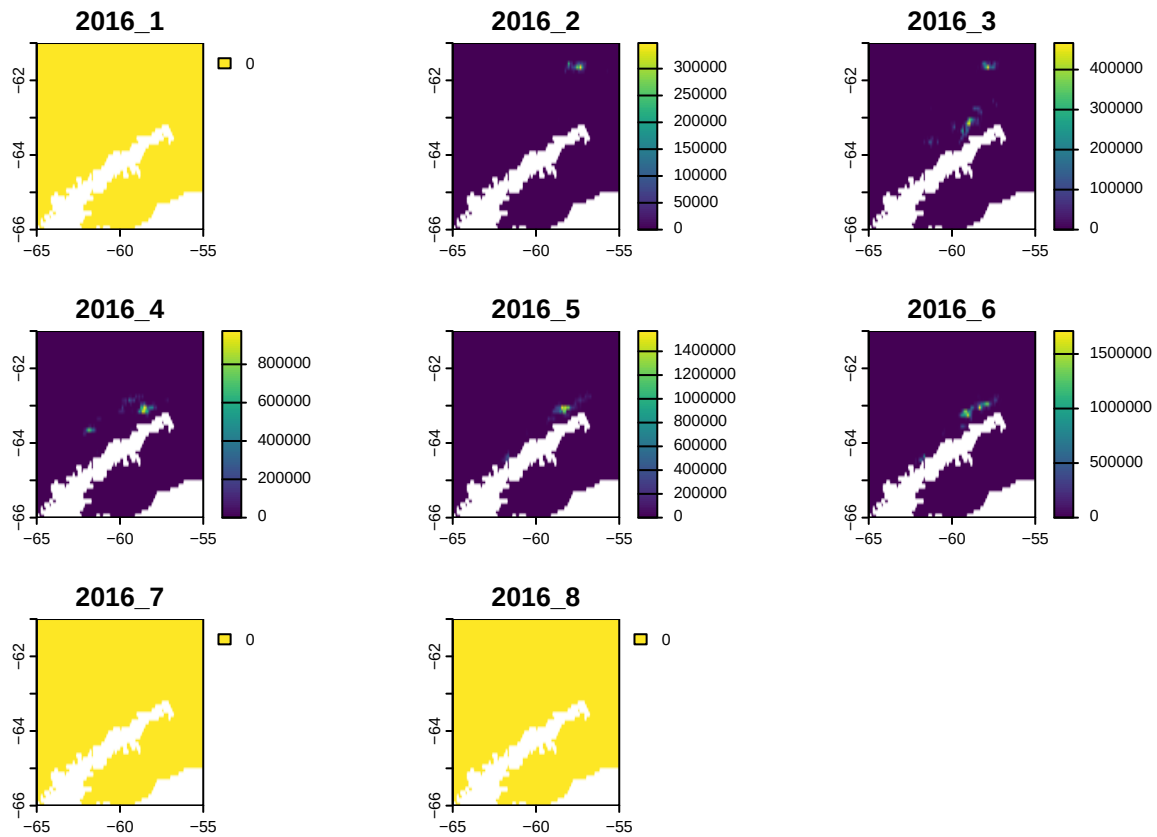
The rasters have a much larger extent than our area of interest, so we'll start out by creating our own extent, which we will then use to crop the rasters.

```
# Create an extent by defining the four corners
min_x <- -65
max_x <- -55
min_y <- -66
max_y <- -61

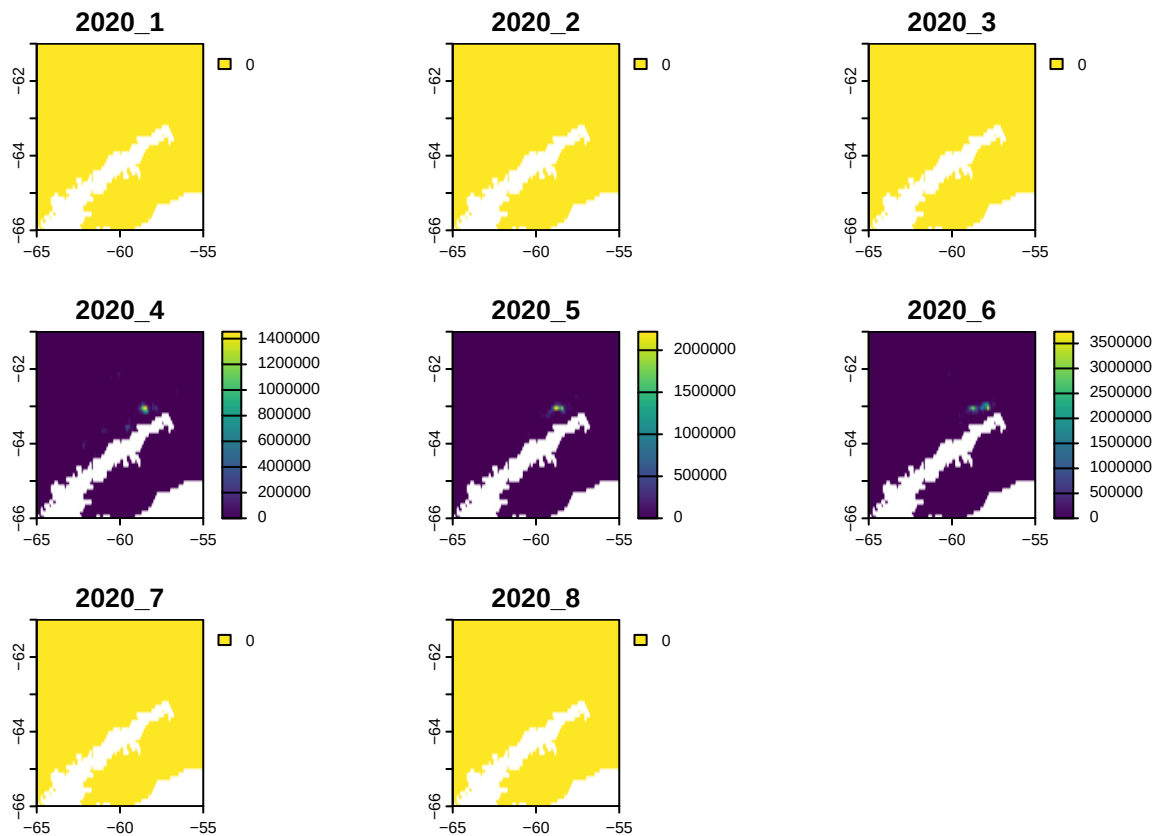
# Combine the four corners into a vector
our_extent <- c(min_x, max_x, min_y, max_y)

# Crop the rasters to our extent
fish_2016 <- crop(fish_2016, our_extent)
fish_2020 <- crop(fish_2020, our_extent)

# Plot the cropped rasters
plot(fish_2016)
```



`plot(fish_2020)`

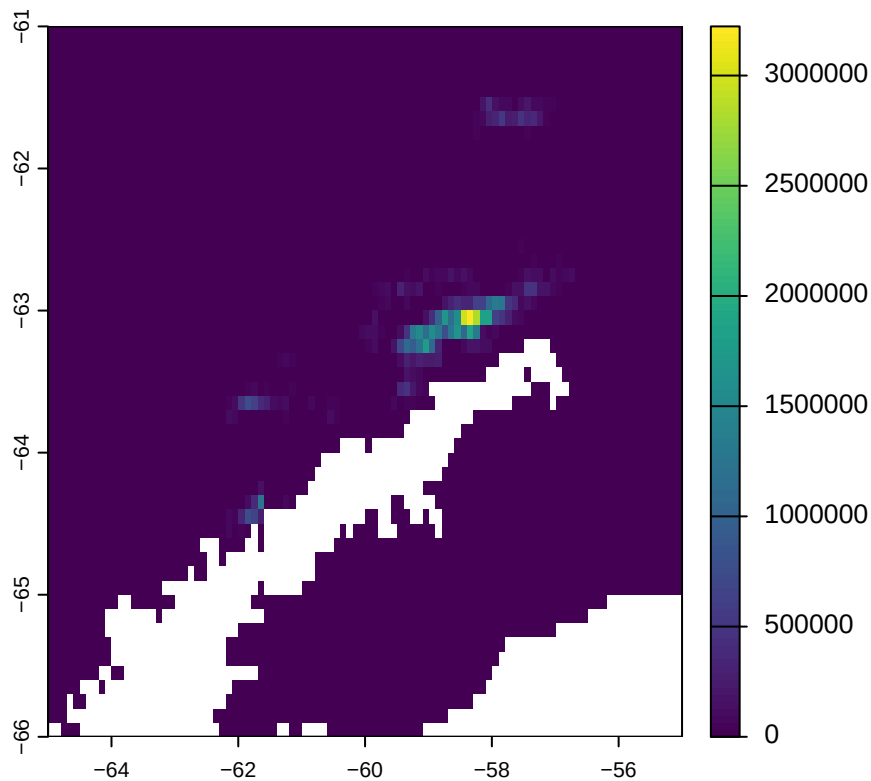


For our first example of looking at seascape change – two time points - let's first calculate the total catch in each pixel for the 2016 and 2020 seasons. We do that simply by adding together (pixel-wise) the values for each month (layer). In this case we use the `sum` function, which adds together all the values in each pixel.

```
catch_2016 <- sum(fish_2016)
nlyr(catch_2016) # notice that there is only one layer now
```

```
## [1] 1
```

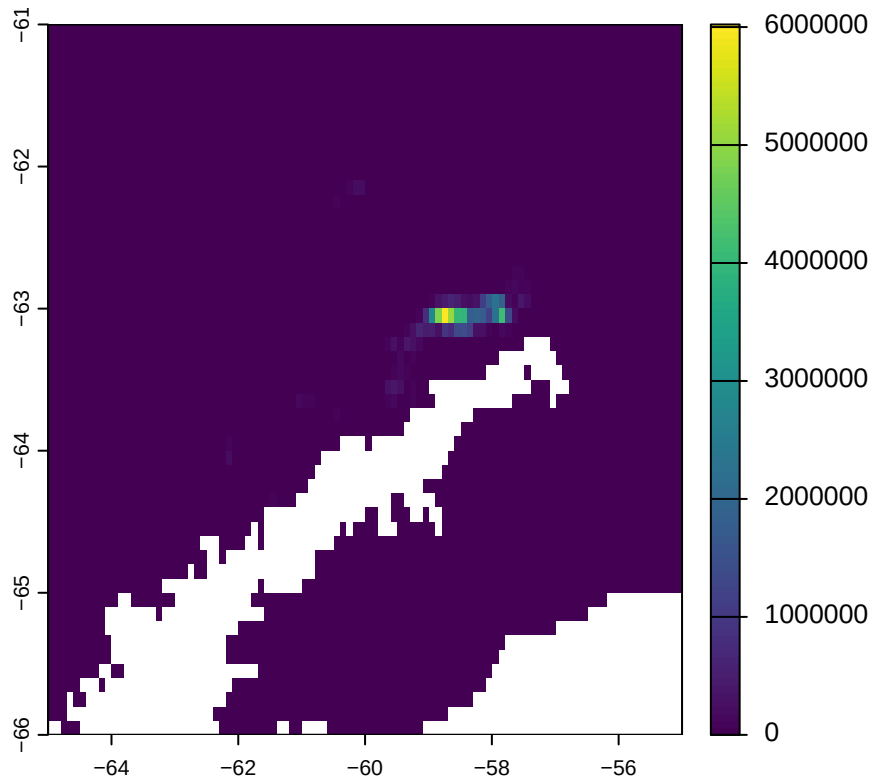
```
plot(catch_2016)
```



```
catch_2020 <- sum(fish_2020)
nlyr(catch_2020) # notice that there is only one layer now
```

```
## [1] 1
```

```
plot(catch_2020)
```

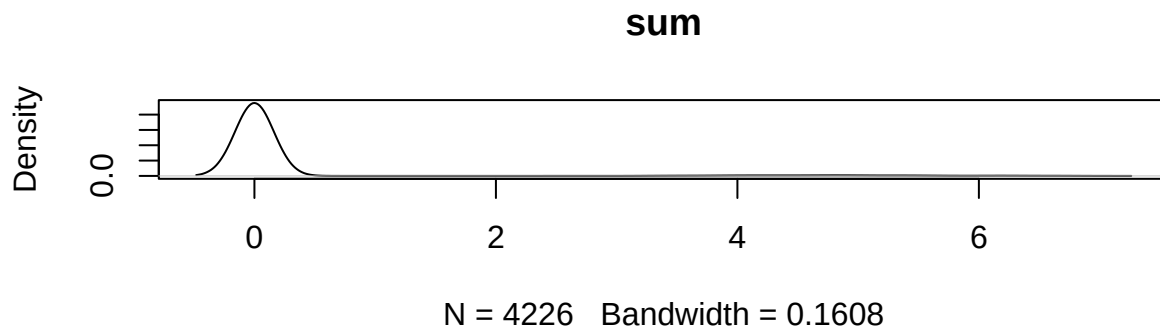
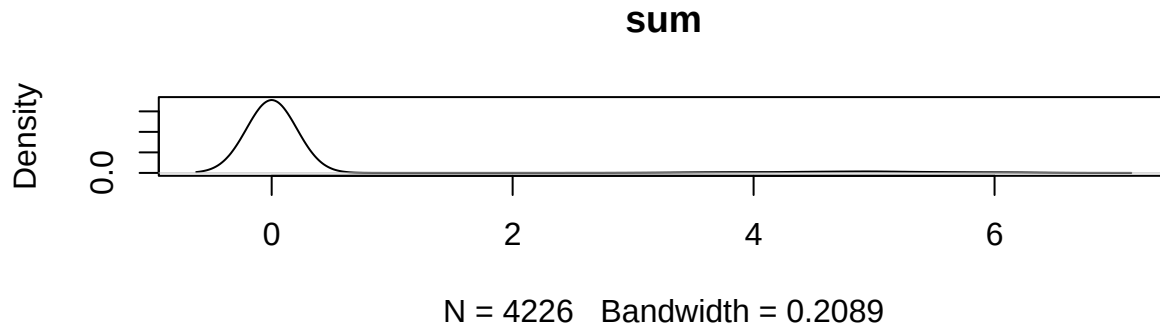


So, these layers are now the total fishing catch over all the months in 2016 and 2020. The distribution of values is highly skewed and hard to see, so let's take the $\log_{10}$ of the values. We first add 1 to each pixel because the $\log_{10}$ of zero is not defined.

```
log_catch_2016 <- log10(catch_2016 + 1) # add 1 to each pixel and calculate the log10
log_catch_2020 <- log10(catch_2020 + 1) # add 1 to each pixel and calculate the log10
```

We can look at the distribution (histogram) of fishing catch values in each raster, first using a rudimentary approach, and then with a better approach in ggplot2.

```
# A simple look
par(mfrow = c(2, 1)) # split the plotting window into 2 columns, 1 row
terra::density(log_catch_2016)
terra::density(log_catch_2020)
```



Let's try and get a better plot in `ggplot2`.

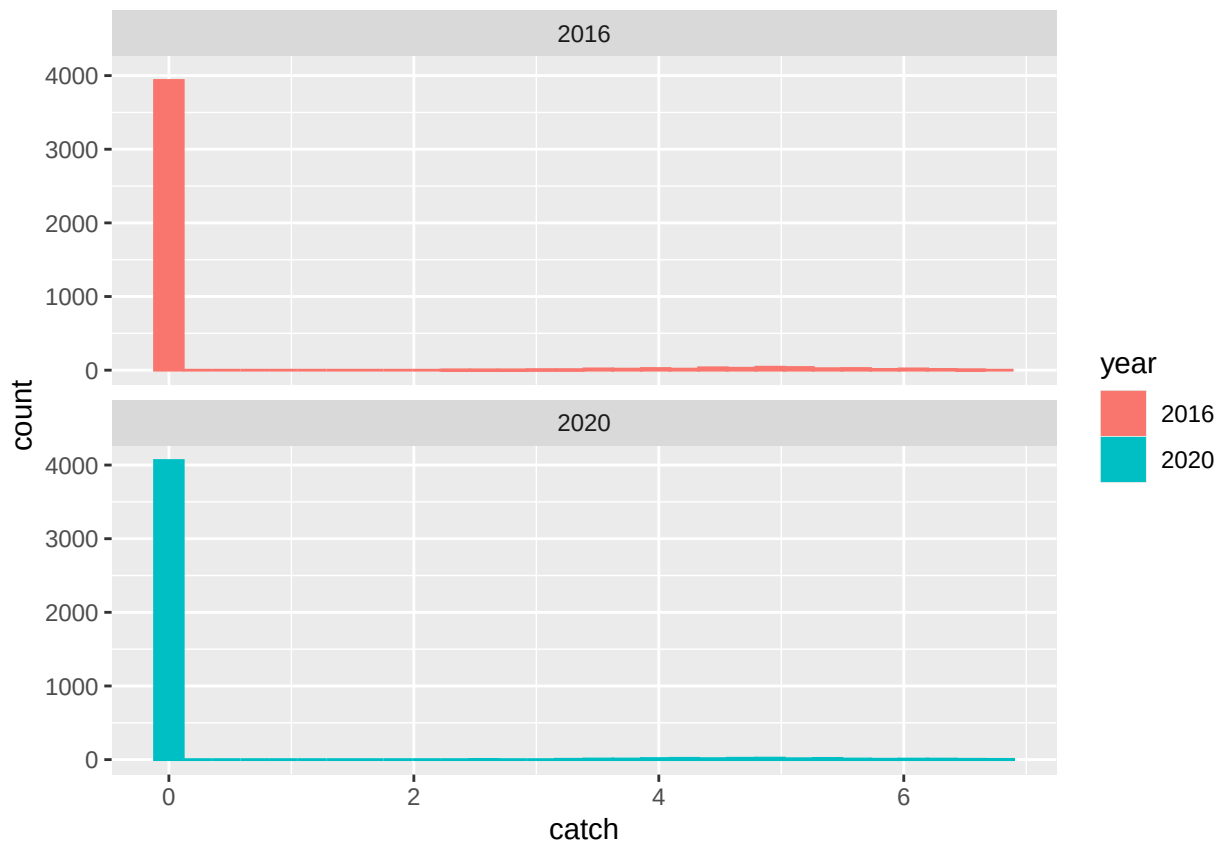
```
# First, we get the values from each raster, using the values() function
values_2016 <- values(log_catch_2016)
values_2020 <- values(log_catch_2020)

# Now these are just vectors of values (they are no longer spatial)
# We want to combine these vectors into a dataframe for display in ggplot2
# First we create a dataframe for each year, filling it with the values
values_2016_df <- data.frame("catch" = as.numeric(values_2016),
                             "year" = "2016")
values_2020_df <- data.frame("catch" = as.numeric(values_2020),
                             "year" = "2020")

# Then, bind the two dataframes by row - 'rbind'
values_df <- rbind(values_2016_df, values_2020_df)

# And then we plot the dataframe containing both years' values
ggplot(data = values_df, aes(x = catch, colour = year, fill = year, group = year)) +
  geom_histogram() +
  facet_wrap(~year, ncol = 1) # this layer produces the 'facets'
```

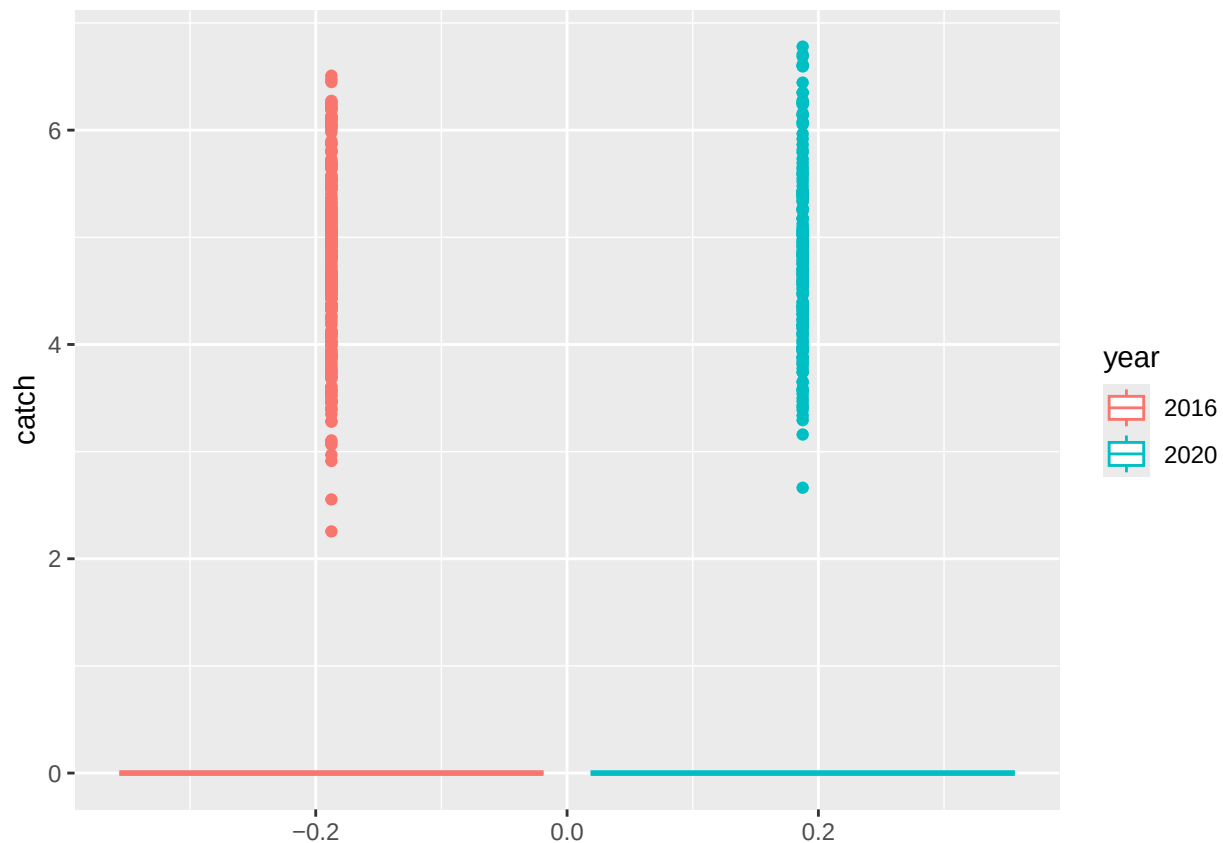
```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
## Warning: Removed 1548 rows containing non-finite outside the scale range
## (`stat_bin()`).
```

It's tricky to compare the distributions because they are so skewed. We can try boxplots, but they don't help much in this case.

```
ggplot(data = values_df, aes(y = catch, colour = year, group = year)) +  
  geom_boxplot()
```

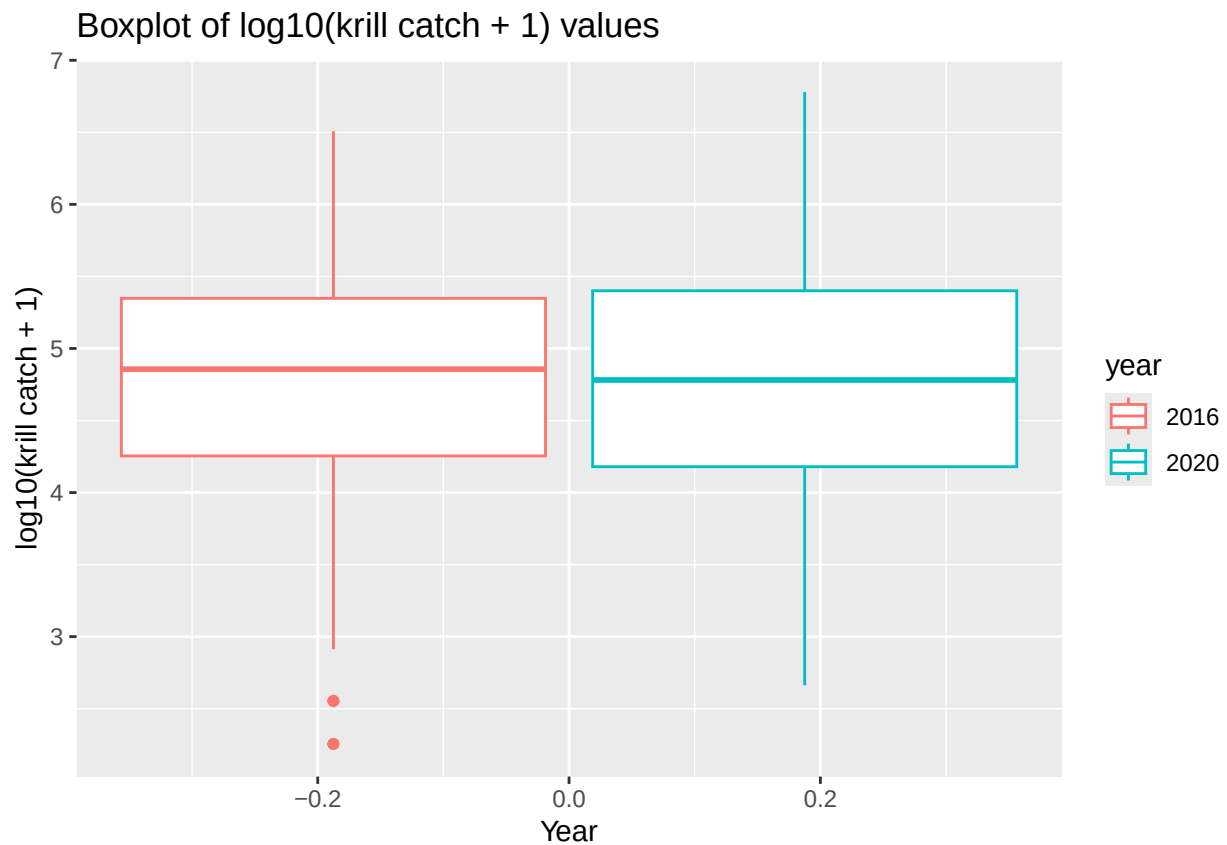
```
## Warning: Removed 1548 rows containing non-finite outside the scale range  
## (`stat_boxplot()`).
```



Let's remove all the pixels with zero catch, and then try the histogram again. Remember, we had already $\log_{10}$ transformed the data after adding 1 to each pixel, so our zero values are now 1 in the data, because $\log_{10}(0 + 1) = 0$. So, let's filter out all zero values, and plot the boxplot again.

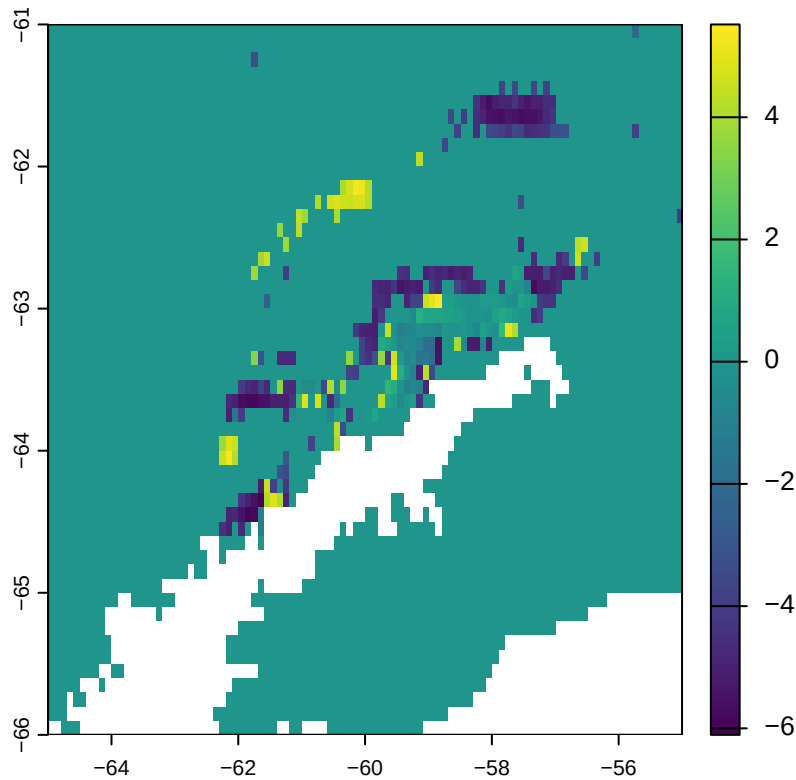
```
values_df <- dplyr::filter(values_df, catch > 0)

ggplot(data = values_df, aes(y = catch, colour = year, group = year)) +
  geom_boxplot() +
  labs(title = "Boxplot of log10(krill catch + 1) values",
        y = "log10(krill catch + 1)",
        x = "Year")
```



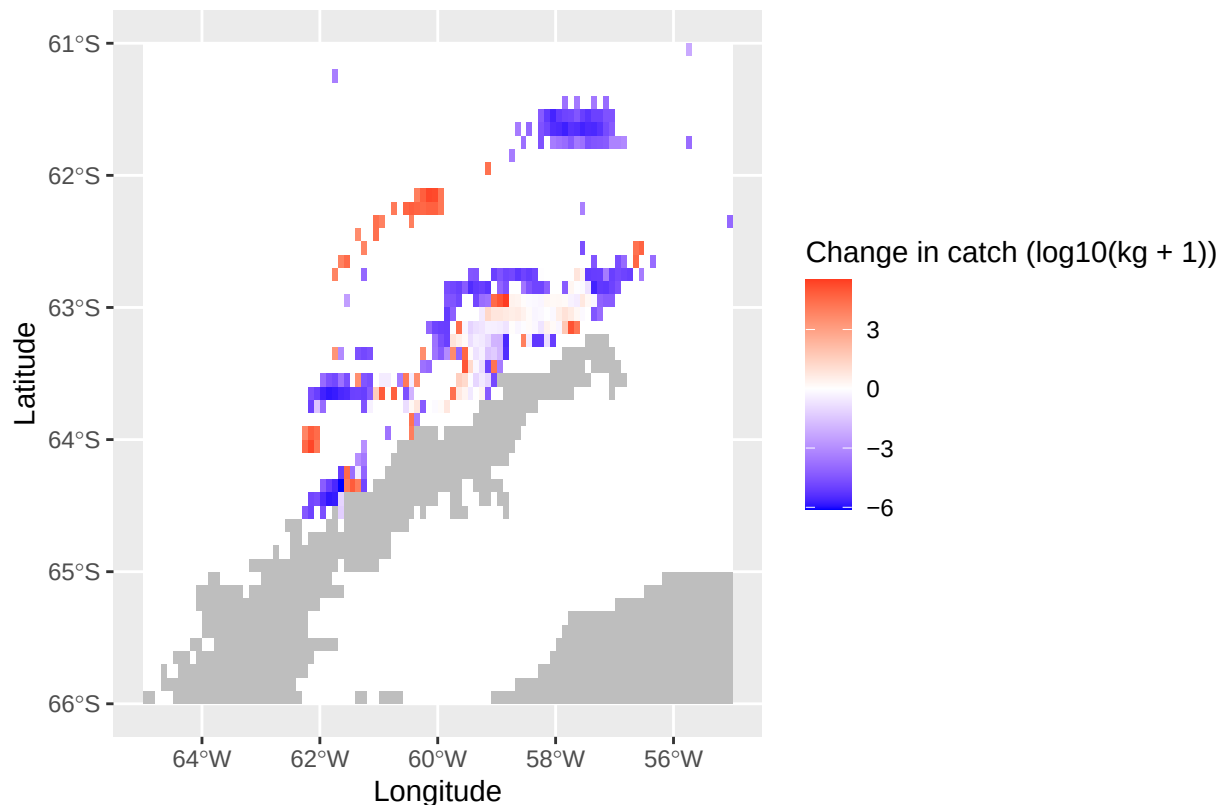
Now, we can compare the two rasters pixel-wise, simply by taking the difference between them (subtracting one raster from the other). Remember from earlier labs, you can do arithmetic operations on rasters directly, and the operations are done pixel-wise.

```
catch_change <- log_catch_2020 - log_catch_2016  
plot(catch_change)
```



We can improve the colour scale, but it takes a bit of work. It's simpler to plot this in `ggplot2`, with the help of the `tidyterra` package. The `geom_spatraster()` geometry is what we use to plot a terra raster in `ggplot2`. We'll also use a spatial colour ramp – `scale_fill_gradient2()` – to get a colour scale with blue for low and red for high values.

```
ggplot() +
  geom_spatraster(data = catch_change) +
  scale_fill_gradient2(
    low = "blue",
    mid = "white",
    high = "red",
    na.value = "grey",
    name = "Change in catch (log10(kg + 1))"
  ) +
  labs(main = "Change in krill catch",
       sub = "2016 to 2020",
       x = "Longitude",
       y = "Latitude")
```



Several points in time

Let's now extend this analysis to more than two points in time. We can use one of the rasters we read in earlier, treating each layer (month) as a point in time. Recall that those rasters each have 8 layers, corresponding with 8 months or 8 time points.

Let's look at the 2020 catch:

```
nlyr(fish_2020)
```

```
## [1] 8
```

Remember, the values are highly skewed, so let's work with log10 from now on.

```
fish_2016 <- log10(fish_2016 + 1)
```

```
fish_2020 <- log10(fish_2020 + 1)
```

First, let's compare the values in each time period, like we did above, but for the 8 months. You'll notice that in this chunk we convert a dataframe from a 'wide' to a 'long' format. The 'long' format is preferred by ggplot and the tidyverse packages, and many people find it easier to work with. Wickham et al. call this format 'tidy data': <https://r4ds.had.co.nz/tidy-data.html>. They discuss 'pivoting' - changing from wide to long - here: <https://r4ds.had.co.nz/tidy-data.html#pivoting>. We'll use the `pivot_longer()` function from the `tidyr` package (https://tidyr.tidyverse.org/reference/pivot_longer.html).

```
# Get the values from the raster
```

```
fish_month_values <- values(fish_2020)
```

```
# Notice, that this is a matrix rather than a vector,
```

```
# because we have 8 layers in the raster
```

```
class(fish_month_values)
```

```
## [1] "matrix" "array"
# Covert the matrix into a dataframe
fish_month_values <- as.data.frame(fish_month_values)

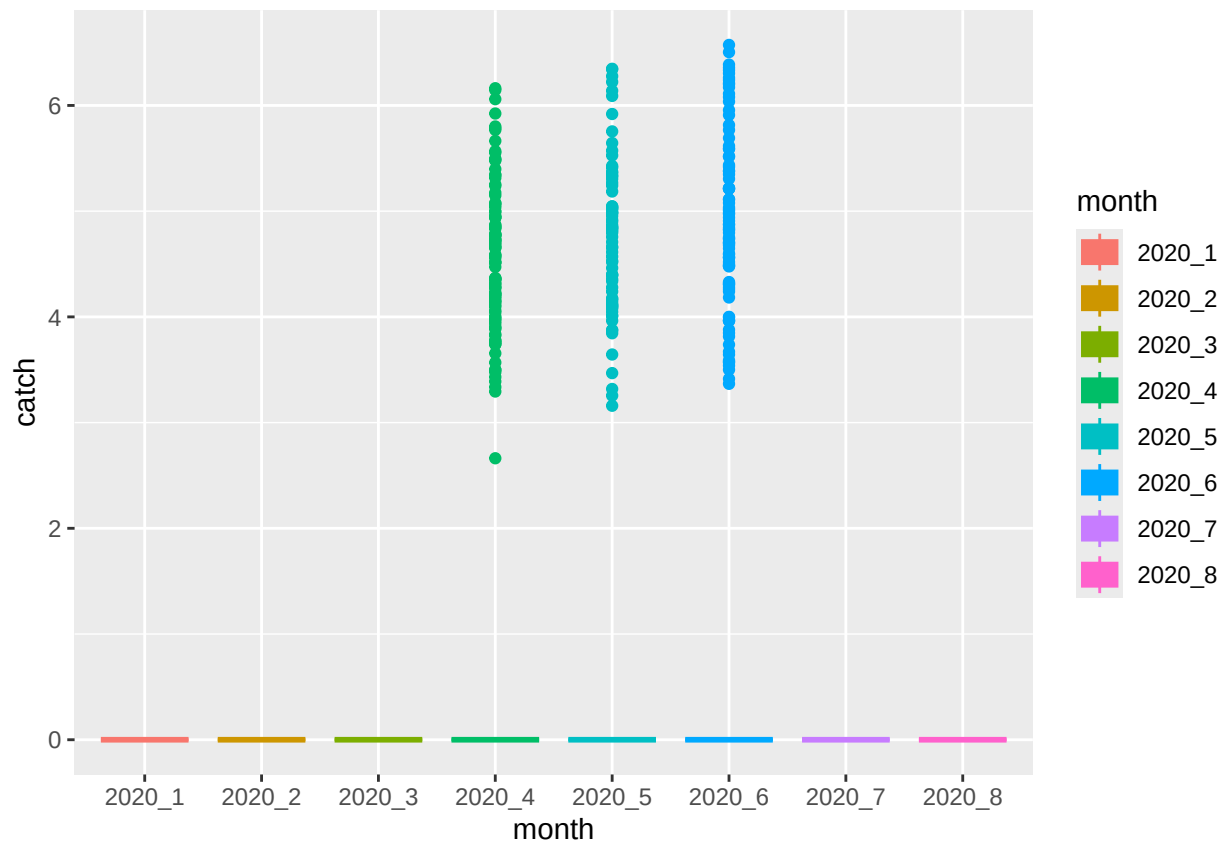
# Change from 'wide' (lots of columns) to 'long' (only a few columns) format for plotting in ggplot
# Take some time to read the pivot_longer help,
# to understand what's happening here
# Note the use of the function everything() to select all columns
# names_to and values_to are arguments that tell the function what to call the new columns
fish_month_values <- pivot_longer(fish_month_values,
                                  cols = everything(),
                                  names_to = "month", values_to = "catch")

# Look at the structure now
head(fish_month_values)

# Remember the months refer to month within a fishing season, so '1' is
# December, not January

# Now we can plot this dataframe
ggplot(data = fish_month_values, aes(y = catch, x = month, colour = month, fill = month, group = month))
  geom_boxplot()

## Warning: Removed 6192 rows containing non-finite outside the scale range
## (`stat_boxplot()`).
```



Despite the distribution still being skewed even after taking the log (the 'boxes' in the boxplots are all pretty much the same), we can see lots of outliers in months 4, 5 and 6 of the season (so, March, April and May),

with a peak in high catches in May ('2020_5'). If we wanted to, we could use the technique as above to filter out the zero values from the dataframe.

Let's assume we want to fit some kind of trendline across the months (the trend in fishing over the year). The first thing we need to do is give our dataframe integer values for the months (currently they are character values). We'll use the `mutate()` and `case_when()` functions. I always find this explainer helpful when using the `case_when` function:

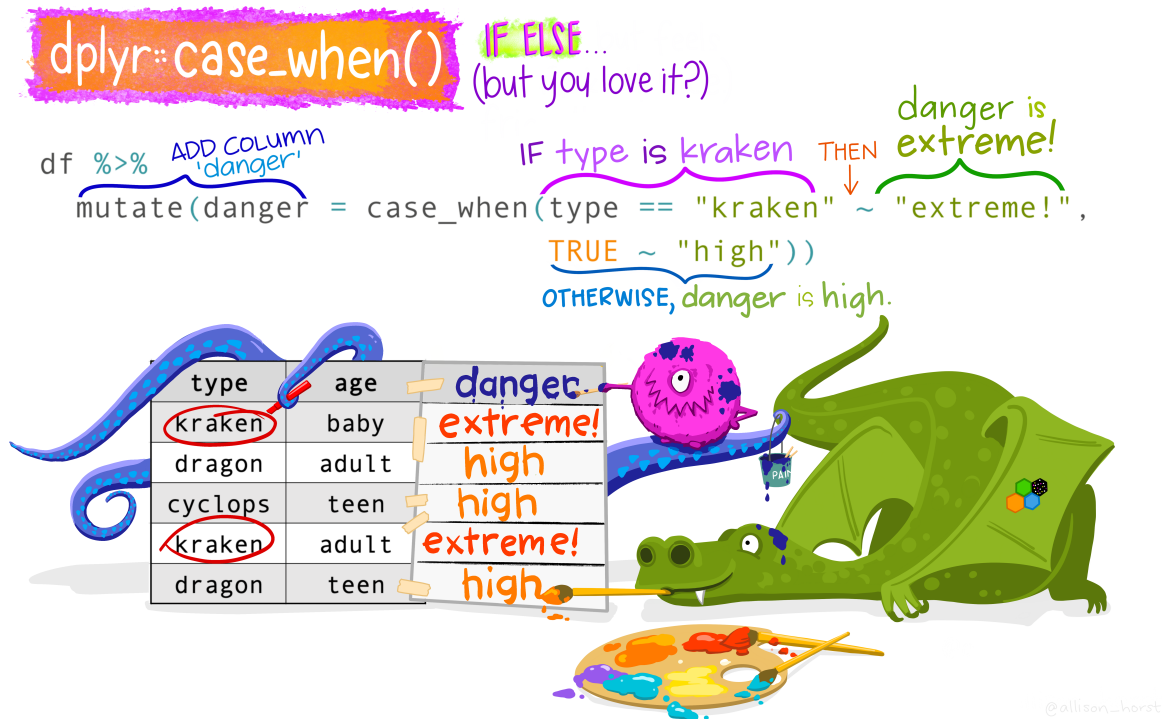


Figure 1: How to use `case_when()`. Artwork by Alison Horst <https://allisonhorst.com/allison-horst>.

We'll use that example to add our new column for month.

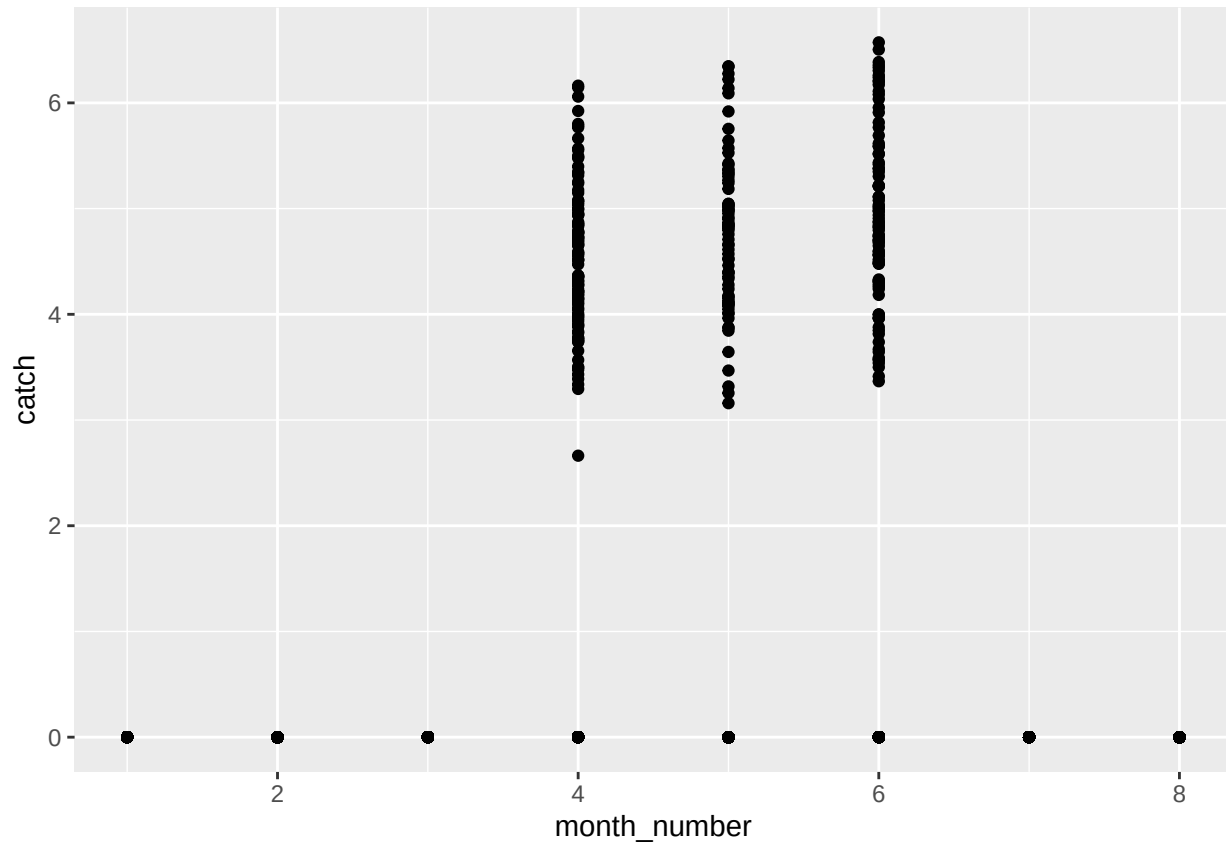
```
fish_month_values <-
  fish_month_values %>%
  mutate(month_number = case_when(month == "2020_1" ~ 1,
                                   month == "2020_2" ~ 2,
                                   month == "2020_3" ~ 3,
                                   month == "2020_4" ~ 4,
                                   month == "2020_5" ~ 5,
                                   month == "2020_6" ~ 6,
                                   month == "2020_7" ~ 7,
                                   month == "2020_8" ~ 8))
```

And now plot it as a scatter plot, fitting a GAM (generalised additive model) trendline.

```
ggplot(data = fish_month_values, aes(x = month_number, y = catch)) +
  geom_point() +
  geom_smooth(method = "gam", method.args = list("k" = 5))

## `geom_smooth()` using formula = 'y ~ s(x, bs = "cs")'
## Warning: Removed 6192 rows containing non-finite outside the scale range
```

```
## (`stat_smooth()`).
## Warning: Failed to fit group -1.
## Caused by error in `data[[txt]]`:
## ! subscript out of bounds
## Warning: Removed 6192 rows containing missing values or values outside the scale range
## (`geom_point()`).
```

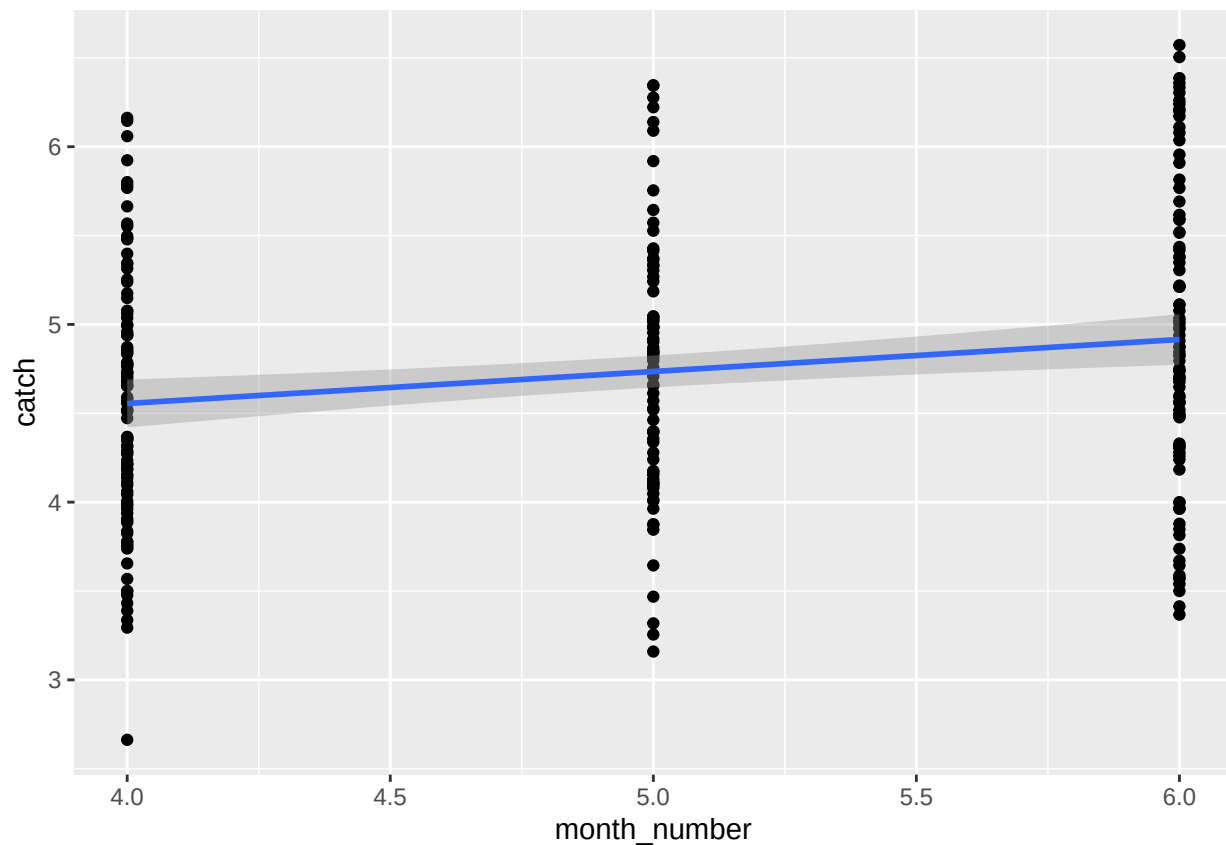


Unfortunately, in this case the computation of the gam smooth fails, probably because there are only three months with data. We can try a linear model instead, setting `method = "lm"`. Let's also filter out the zero values.

```
fish_month_values_no_zeros <- filter(fish_month_values, catch > 0) # Filter out the zeros in catch

ggplot(data = fish_month_values_no_zeros, aes(x = month_number, y = catch)) +
  geom_point() +
  geom_smooth(method = "lm")
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

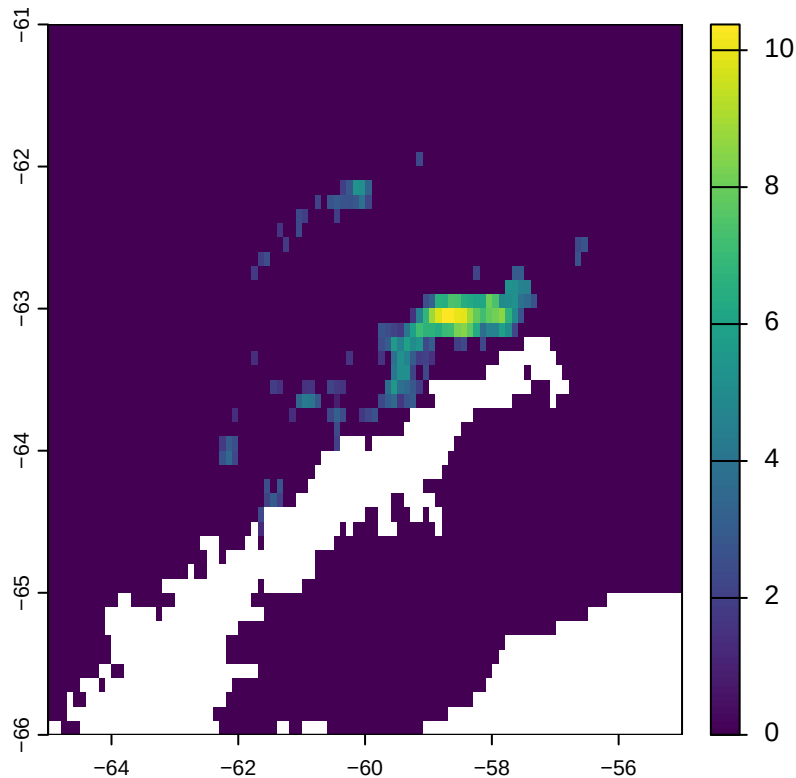



That's better. We can now see that there is a linear increase in catch from April to June.

In the 'try for yourself' section, you can have a go at following these steps with a different dataset...

We can look at a few metrics to tell us a bit about the time series in each pixel. For example, we could look at the variance, to see how much the values in each pixel fluctuate over the time series.

```
fish_2020_variance <- app(fish_2020, fun = "var") # Note use of the 'app' function to 'apply' the speci
plot(fish_2020_variance)
```



In the plot you can see that the area around -63 degrees S and (y-axis) and -58 degrees E (x-axis) has the highest variance in fishing catch over the months.

We could look at the trend in each pixel (rather than the overall trend, as we did above), using two methods.

The first is a spatial version of Kendall's Tau. The function we could use is `tau` from `terra`. This calculates the trend in each pixel. (You will get warnings running this, you can safely ignore them).

```
# layer index as time (1:8)
time <- 1:nlyr(fish_2020)
# run the function
tau <- app(fish_2020, fun = function(x) cor(x, time, method = "kendall"))
```

```
## Warning in cor(x, time, method = "kendall"): the standard deviation is zero
## Warning in cor(x, time, method = "kendall"): the standard deviation is zero
## Warning in cor(x, time, method = "kendall"): the standard deviation is zero
## Warning in cor(x, time, method = "kendall"): the standard deviation is zero
## Warning in cor(x, time, method = "kendall"): the standard deviation is zero
## Warning in cor(x, time, method = "kendall"): the standard deviation is zero
## Warning in cor(x, time, method = "kendall"): the standard deviation is zero
## Warning in cor(x, time, method = "kendall"): the standard deviation is zero
## Warning in cor(x, time, method = "kendall"): the standard deviation is zero
## Warning in cor(x, time, method = "kendall"): the standard deviation is zero
## Warning in cor(x, time, method = "kendall"): the standard deviation is zero
## Warning in cor(x, time, method = "kendall"): the standard deviation is zero
## Warning in cor(x, time, method = "kendall"): the standard deviation is zero
## Warning in cor(x, time, method = "kendall"): the standard deviation is zero
## Warning in cor(x, time, method = "kendall"): the standard deviation is zero
## Warning in cor(x, time, method = "kendall"): the standard deviation is zero
## Warning in cor(x, time, method = "kendall"): the standard deviation is zero
## Warning in cor(x, time, method = "kendall"): the standard deviation is zero
```

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

[illegible]

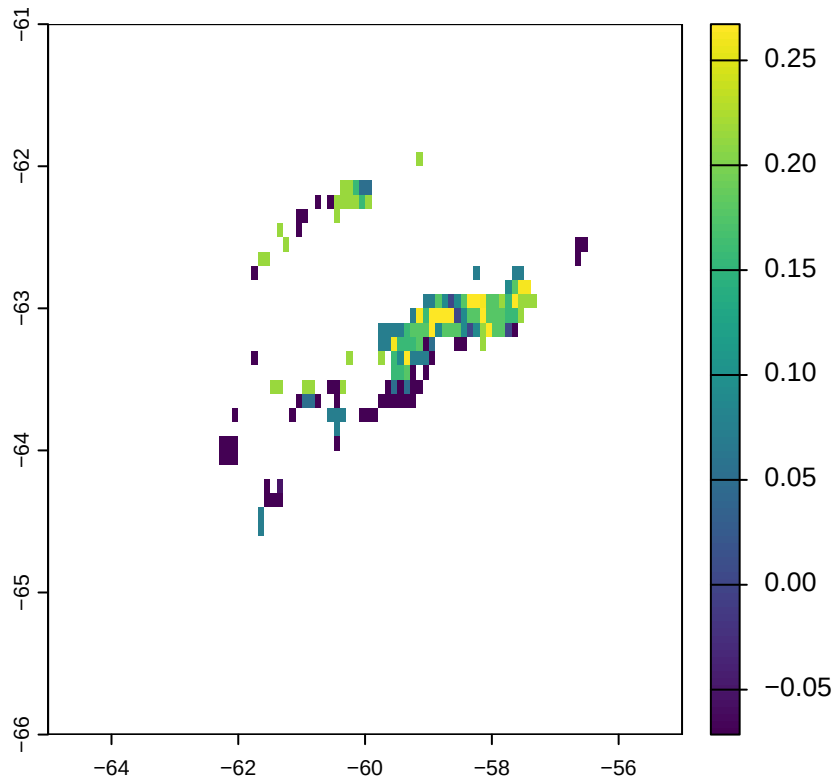
[illegible]

[illegible]

[illegible]

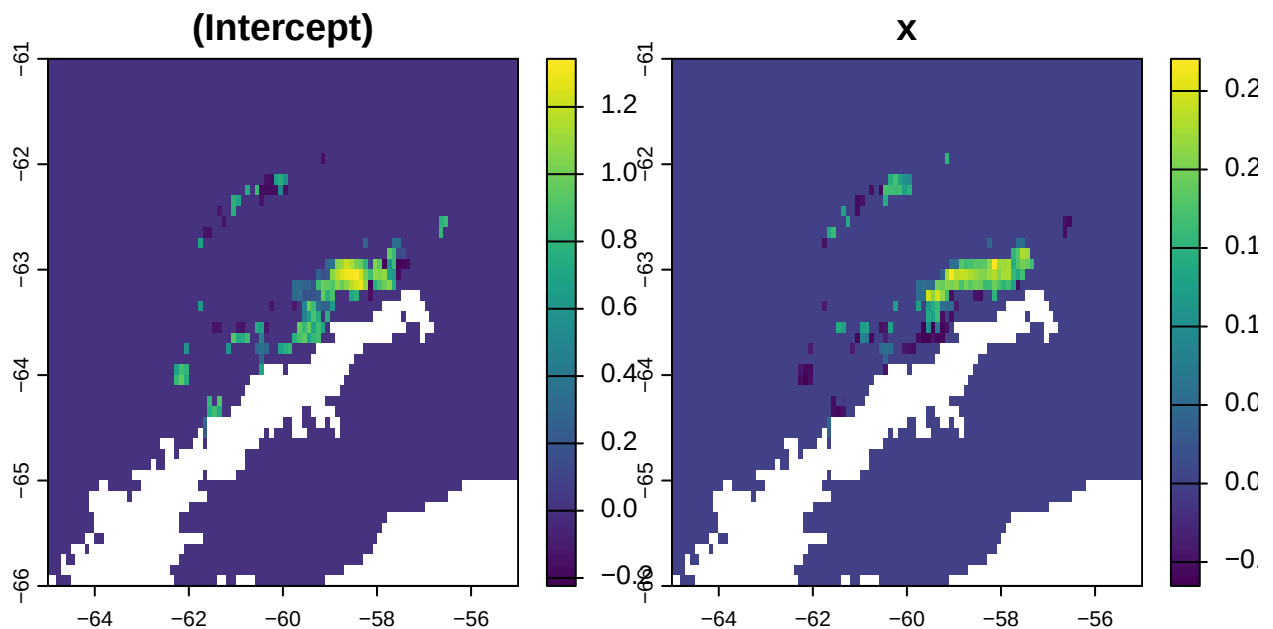
[illegible]

```
# plot the result
plot(tau)
```



We could also use the `regress()` function in `terra`, which runs a linear regression (linear model) for each pixel in the raster. We can then plot the slope and intercept of the regression line.

```
# fish_trend <- kendall(stack(fish_2020)) # This would fail because there are all NA values in the rast
fish_trend <- regress(fish_2020, 1:nlyr(fish_2020)) # This works.
plot(fish_trend)
```



```
# X is the slope of the regression line, and intercept is the intercept of the regression
```

Try for yourself

The last two steps were only moderately successful, because of the kind of data we were using. It was very sparse: many cells had no values. As an exercise, starting in class, and carrying on at home, I'd like you to try and implement the workflow above with a different dataset. This is building towards the kind of analysis you will need to do for your workflow assessment.

I've uploaded a set of sea surface temperature files for the eastern North Pacific to Github. I add the daily sea surface temperature on 15 September for each year from 2010-2020 (11 files). Think Marine Heatwaves... (Oliver et al. 2021).

Here is the first example of how you could access the files, directly, or you can download them from the GitHub repository and read them in from your own computer. Remember to change the path to the file to wherever you have saved it. The folder containing the files is: https://github.com/ryanreisinger/SOES3056/raw/main/Lab_08/Resources/annual_sst/

```
sst_2010 <- rast("https://github.com/ryanreisinger/SOES3056/raw/main/Lab%2008/Resources/annual_sst/sst_2010.tif")
plot(sst_2010)
```

